

Match Day Stadium Simulator

A next-generation 3D renderer for TifoMaker.org — turning the 3D preview into an immersive, realistic match-day choreography experience, without slowing the editor.

Product: TifoMaker.org — 3D stadium tifo / choreography designer

Prepared by: Lead Renderer Architect

Document type: Deep analysis + architecture proposal (no implementation)

Strategic priority: Maximum simulator realism & immersion, with the editor renderer fully protected

Status: For review and direction

In one sentence

Keep today's fast card-instancing renderer exactly as it is for editing, and build a separate, lazy-loaded high-fidelity "Match Day Simulator" on the same Three.js engine and the same seat-map data — so the editor stays instant while the simulator becomes the most realistic fan-choreography visualizer available.

Contents

1. Executive Summary
2. Current System Analysis (architecture, pipeline, bottlenecks, limits)
3. Reuse / Replace / Keep-Lightweight
4. Vision: the Match Day Stadium Simulator
5. Dual-Renderer Architecture (how both coexist)
6. Rendering Technology Analysis (Options A–E + recommendation)
7. Quality Levels & Automatic Performance Scaling
8. Stadium Structure Improvements
9. Realistic Stadium Materials
10. Match Day Atmosphere: the Crowd System
11. Banners & Flags
12. Choreography Elements
13. Match Day Effects
14. Viewing Modes & the Camera Director
15. Competitive Analysis
16. Development Phases & Estimated Complexity
17. Performance & Visual Quality Expectations
18. Risks & Mitigations
19. Long-Term Roadmap
20. Appendix: Data Contracts & Module Layout

1. Executive Summary

The current 3D view is a small, well-built engineering achievement: the entire renderer is a single file (preview3d.ts) that draws every seat as one GPU-instanced quad colored directly from the same buffer the 2D editor paints into. It is fast, cheap, and reuses across the editor, the landing-page hero, the public share and community viewers, and the GIF/MP4 export pipeline. Its weakness is not performance — it is realism. There are no stands, no roof, no crowd, no lights, and no shadows; the colored cards float in space above a flat green plane. That is why it reads as a “preview” rather than a stadium.

This proposal recommends a dual-renderer strategy. The existing renderer is kept untouched as the fast EDITOR renderer (quality tiers LOW and MEDIUM). A new, separate, lazy-loaded SIMULATOR renderer (tiers HIGH and ULTRA) is built on the same Three.js engine and the same SeatMap / DesignStore data layer, adding real stand architecture, PBR materials, a filled crowd, banners and flags, floodlights and shadows, atmospheric effects, post-processing, and a cinematic camera system. Because the simulator is a separate module loaded only when the user enters Match Day mode, the editor's speed, bundle size, and GPU budget are never affected.

Recommendation at a glance

Engine: stay on Three.js (already in use at r184). Do not migrate to Babylon.js — the migration cost is high and buys no decisive advantage over a modern Three.js scene.

Architecture: one engine, two renderers. Editor renderer = current file, unchanged. Simulator renderer = new lazy-loaded module sharing the data layer.

Sequence: ship realism in phases, each a visible win, starting with stand architecture + lighting (the single biggest “it's a real stadium now” jump), then crowd, then pitch-side detail, banners/flags, effects, and cinematic cameras.

Result: design -> simulate -> film -> share, all seat-accurate — a combination no competing tool offers.

2. Current System Analysis

2.1 Architecture

The 3D system is intentionally minimal and lives almost entirely in **src/render/preview3d.ts** (Three.js r0.184 + OrbitControls). One class, Preview3D, owns the scene:

- **Seats as one InstancedMesh.** A single flat card quad (PlaneGeometry 0.45 m x 0.7 m, unlit MeshBasicMaterial, double-sided) is instanced once per seat. Per-instance transforms come from SeatMap.pos3 (metric world positions precomputed by the seat-map generator off the main thread); each card is positioned at chest height, turned to face the bowl center, and leaned back with the rake.
- **Color is zero-copy from the design.** Per-instance colors are read straight from DesignStore.cells (one palette index per seat). There is no sync step: the store's dirty callback recolors only the changed instances, and a palette-change callback rebuilds the color cache. The 3D view and the 2D editor are therefore always identical by construction.
- **Environment is three primitives.** A dark circular apron, a flat green pitch plane (105 x 68 m), and a few white line primitives (touchlines, halfway line, center circle). All unlit.
- **Camera.** A 50-degree PerspectiveCamera with OrbitControls and five named presets (TV gantry, Behind goal, Pitch level, Aerial, Full view). The buffer is kept readable (preserveDrawingBuffer) so the exporter can grab frames; renderOnce() steps a single frame for video/GIF capture.
- **Choreography + realism aids.** applyReveal() lerps each card between dark and full color so reveals play identically in 2D and 3D; a per-session no-show mask greys ~10% of seats on demand.

Crucially, Preview3D is **reused in nine places** — the editor split view, the AI panel, the toolbar, the public share page, the community feed, the standalone viewer, the GIF/MP4 export, and the landing-page hero. Any change to this file ripples across all of them, which is the central reason the simulator should be additive and separate rather than a rewrite of this file.

2.2 Rendering pipeline

The per-frame pipeline is about as lean as WebGL gets:

- requestAnimationFrame loop -> controls.update() -> renderer.render(scene, camera).
- All seats are submitted in a single instanced draw call; the pitch and lines add only a handful more. No lighting, no shadow pass, no post-processing.
- Edit data flow: a 2D paint stroke updates DesignStore.cells, which emits a dirty event with the changed indices; the renderer calls setColorAt only for those instances and flags the instance-color buffer. Geometry is never rebuilt on edit.

2.3 Performance characteristics & bottlenecks

The design is fast precisely because it does so little. Its real ceilings are about fidelity, not frame rate.

Strengths to preserve

- One instanced draw call scales to 60,000–80,000+ seats at 60 fps on modest hardware, including mobile.
- Unlit material means zero per-pixel lighting cost; tiny GPU memory (one geometry + instance buffers).
- Incremental recolor is O(changed seats), not O(all seats), so editing stays smooth.

Bottlenecks & ceilings (mostly latent today, relevant as fidelity grows)

- A full recolor (palette swap or reveal frame) is an O(n) CPU loop of setColorAt calls. Fine for occasional edits; it would become a cost if animated every frame at ULTRA crowd counts.

- The instanced mesh is a single object, so per-instance frustum culling and level-of-detail do not apply — irrelevant while instances are cheap flat quads, but a real concern once instances become lit 3D seat or crowd meshes.
- `preserveDrawingBuffer = true` (needed for capture) disables a swap optimization — a minor, acceptable cost.
- The hard ceiling is expressive: unlit flat cards cannot convey depth, occlusion, scale, time of day, or atmosphere. No amount of tuning makes this path look like match day.

2.4 Visual limitations

- No stand architecture — the bowl is implied by where cards float, not built as structure, so you never see the stadium as a building.
- No crowd — empty seats are just dark cards; there is no sense of a packed end behind the tifo.
- Unlit and shadowless — a flat, even look with no light direction, no ambient occlusion, no day/night, no floodlights.
- No roof, facade, dugouts, tunnel, technical area, or advertising boards — none of the pitch-side detail that signals “real stadium.”
- Flat quad cards with no thickness or material variation; a single fixed dark environment; static preset cameras with no cinematic motion or broadcast framing.

3. Reuse / Replace / Keep-Lightweight

The data layer is the crown jewel and is fully reused; the editor renderer is kept lightweight and untouched; new fidelity is added in a separate simulator module.

System	Verdict	Rationale
SeatMap (pos3, uv, tier/row/section, neighbors, bounds, templateRef)	REUSE	The geometric source of truth, generated off-thread. The simulator builds stands, crowd, and banners from the same map — guaranteeing the design lines up with the structure.
DesignStore (cells, palette, onDirty, onPaletteChange)	REUSE	Live design data + dirty events. Both renderers subscribe; edits reflect wherever a view is open, with no extra sync.
StadiumTemplate (plan a/b/exponent, tiers, aisles, sectionsPerTier)	REUSE + EXTEND	Parametric bowl definition. Extend (backward-compatibly) with roof, track width, per-stand overrides, pitch-side furniture — absent fields keep today's behavior.
applyReveal contract + export hooks (renderOnce, preserveDrawingBuffer)	REUSE	Choreography reveal and frame capture already work; the simulator implements the same contract so reveals and exports behave identically.
Instancing technique	REUSE / CARRY FORWARD	The one-draw-call pattern is exactly what the crowd, seats, banners, and flags need at scale.
preview3d.ts renderer	KEEP LIGHTWEIGHT (do not touch)	It is the editor / hero / share / community / export renderer and the LOW/MEDIUM tier. Only optional, flag-gated touches — never anything that risks editor speed.
Environment (apron, flat pitch, line primitives)	REPLACE (simulator only)	Upgraded in the simulator to a full pitch, pitch-side furniture, stands, roof, and facade. The editor keeps the cheap version.
Camera presets (5 static shots)	EXTEND	Become the seed for a Camera Director with motion, broadcast framing, and a flyover spline in the simulator.

4. Vision: the Match Day Stadium Simulator

The goal is a mode the user enters after designing a tifo, where the same seat-accurate design is shown on a believable, floodlit bowl at dusk — packed with a color-coordinated crowd, flags waving on the rails, smoke drifting across the ultras' end, the choreography blazing across the painted stand — and viewed from a broadcast gantry, a seat in the crowd, pitch level, or a drone. The feeling target is simple: “this looks like the real thing on match day.”

Realism here means web-real, not offline-CGI: physically based materials, real lighting and shadows, a dense instanced crowd, and tasteful post-processing — a clean, convincing look that runs in a browser, scales down gracefully, and never compromises the editing experience.

5. Dual-Renderer Architecture

One engine, one data layer, two renderers — only ever one heavy GPU context active at a time.

5.1 How both coexist

- **Shared data layer (single source of truth):** SeatMap + DesignStore + StadiumTemplate. Neither renderer owns design data; both read from it.
- **Editor renderer (preview3d.ts):** unchanged. Mounted in the editor split view, hero, share, community, and export. Instant and light.
- **Simulator renderer (new):** a separate module under src/render/simulator/, dynamically imported only when the user clicks “Match Day Simulator.” Its heavy code, textures, crowd assets, and post-processing ship in a separate bundle chunk — the editor never downloads or runs them.
- **Handoff:** entering the simulator passes the current SeatMap, DesignStore, and a camera intent; leaving it fully disposes the GPU resources (the existing renderer already models this discipline well, which we carry over to avoid context leaks).
- **Quality routing:** LOW/MEDIUM open the editor renderer; HIGH/ULTRA open the simulator. A capability probe picks a sensible default and can auto-downscale.

Why separate beats “one clever renderer”

Putting lights, shadows, crowd, and post-processing into the shared preview3d.ts would tax the editor and the eight other places it is reused — the exact opposite of the stated priority. A separate, lazy-loaded module isolates all that weight behind a single entry point, while one shared engine (Three.js) keeps the codebase, types, and mental model unified.

6. Rendering Technology Analysis

Five options were evaluated against performance, visual quality, development effort, and long-term maintainability.

Option	Performance	Visual quality	Dev effort	Maintainability	Verdict
A. Enhance existing renderer	Risks editor speed (shared path)	Medium ceiling	Low–Med	Muddies the fast path	Partial — MEDIUM only
B. Modern Three.js (new scene)	Excellent, full control	High (PBR, post, particles)	Medium (no migration)	High (one engine)	Core of recommendation
C. Babylon.js	Strong, good built-ins	High out-of-box	High (full migration)	Two engines or full rewrite	Reject
D. Hybrid (one engine, two configs)	Editor light, sim heavy	High	Medium	High	Recommended pattern
E. Separate simulator module	Isolated heavy context	High–Very high	Medium–High	High (clean boundary)	Recommended pattern

6.1 Recommendation

Adopt **D + E built on B**: keep Three.js, and build a separate, lazy-loaded simulator module (Option E) that is a heavy configuration of the same engine (Option D), using modern Three.js features (Option B). Reject a Babylon.js migration (Option C): it carries a full rewrite or a two-engine burden with no decisive advantage over a well-built Three.js scene. Reject Option A as the primary strategy because adding lights and shadows to the shared renderer would slow the editor — though small, flag-gated touches there can serve the MEDIUM tier.

Future-proofing

Three.js is shipping a WebGPU backend. Design the simulator's scene-building behind a thin abstraction so a future WebGPURenderer swap is possible for extra headroom without rewriting how stands, crowd, and effects are constructed. This is an optional long-term lever, not a near-term dependency.

7. Quality Levels & Automatic Performance Scaling

Tier	Renderer	What it adds	Target
LOW	Existing (preview3d.ts)	Today's cards + flat pitch. Instant, mobile-safe.	60 fps everywhere
MEDIUM	Existing + light touches	Cheap extruded bowl shell, one directional light (no shadows), subtle empty-seat shading.	60 fps desktop / good mobile
HIGH	Simulator	Real stands + roof, PBR materials, sun + one shadow light, dusk sky, instanced impostor crowd, floodlights.	50–60 fps desktop / 30 fps mid mobile
ULTRA	Simulator + full stack	HIGH plus post-processing (bloom, SSAO, tone mapping, AA), GPU particle effects (smoke, pyro, confetti), waving cloth flags/banners, cinematic camera moves.	30–60 fps high-end desktop

7.1 Automatic scaling

- On entry, probe the GPU (renderer info, max texture size), device pixel ratio, and a short frame-time sample to pick a default tier.
- If the running frame-time budget is exceeded, step down automatically: drop post-processing first, then particle budgets, then crowd density and 3D-figure LOD, then shadow-map resolution, then device pixel ratio.
- Expose an explicit Quality control (Auto / Low / Medium / High / Ultra). Respect prefers-reduced-motion and low-battery hints by defaulting conservative.
- LOW is always the guaranteed floor — the editor renderer already runs anywhere, so there is never a “black screen” outcome.

8. Stadium Structure Improvements

The bowl is already parametric (a superellipse plan with per-tier rows, rake, depth, elevation, plus aisles and sections), and the seat map already encodes tiers, rows, sections, and normalized uv. That means real structure can be extruded from the same parameters that place the seats — the design and the building stay perfectly aligned. Most items below are backward-compatible template extensions: absent fields reproduce today's uniform bowl.

Capability	Data / template change	Renderer work	Complexity
Stand shell (concrete bowl behind seats)	Reuse existing ring geometry per tier	Loft/extrude a mesh per tier from the superellipse rings	L
Football vs running-track	Add trackWidthMeters (pitch-to-stand offset)	Offset the front of the bowl; draw track surface	S
Pitch-side / front-row seating	Front tier at -0 offset and low elevation	Place a low front block close to the touchline	S
Modern bowl / historic / multi-tier	Per-tier configs (already supported)	Vary rake, depth, gaps, materials	S–M
Single-tier ultra wall + asymmetry	Optional per-stand (E/N/W/S) overrides	Build each side independently (steep wall on one end, etc.)	M
Custom geometry	Extend the existing custom-stadium schema	Simulator reads the new structural fields	S
Roof structures (none / cantilever / full / partial)	Per-stand roof type + depth	Truss + panels; casts shadow, encloses light/atmosphere	M–L
Dugouts, tunnel, technical area	Anchor at halfway / chosen stand	Low-poly placeable furniture from pitch bounds	S–M
Advertising boards (LED ring)	Pitch perimeter flag + optional text/texture	Instanced boards; can show user text	S

9. Realistic Stadium Materials

In the simulator, surfaces move to physically based materials (MeshStandardMaterial / MeshPhysicalMaterial) lit by real lights. Empty seats become an instanced low-poly seat mesh; occupied seats show the colored tifo card or a crowd figure. Two instanced meshes (seats and cards/crowd) keep draw calls flat.

Material / feature	Technique	GPU cost	Tier
Seats (empty)	Instanced low-poly mesh, PBR albedo/roughness	Low–Med	HIGH
Concrete (stands, stairs, entrances)	Tiled albedo + normal + roughness, low metalness	Low	HIGH
Railings	Thin instanced metal (metalness)	Low	HIGH
Roof / truss (metal)	Standard metal material + simple truss geo	Low–Med	HIGH
Glass (facade, VIP)	Physical material with transmission	High	ULTRA only
Stairways / tunnels	Modeled from the concrete material set	Low	HIGH
Lighting + 1 shadow-casting light	Directional sun/floodlight + shadow map	Med	HIGH
Texture pipeline	Small shared atlas, KTX2/Basis compression, lazy-loaded	One-time	HIGH

The decisive perceived-quality jump is moving from unlit to **PBR plus one shadow-casting light**; transmission glass and many dynamic lights are reserved for ULTRA. A small, curated, compressed texture set keeps the simulator chunk lean and is downloaded only on entry.

10. Match Day Atmosphere: the Crowd System

A filled crowd is the single biggest atmosphere lever. Three approaches, mixed by level of detail:

Approach	Realism	Cost	Tier / use
Billboard impostors (camera-facing textured quads, instanced)	Good at distance	Very low (one instanced draw)	HIGH baseline — far stands
Low-poly instanced 3D figures	High up close	Medium (needs LOD + culling)	ULTRA / near stands
Animated crowd (idle sway, jump, clap)	Very high	Med–High (vertex/texture-sheet)	ULTRA

10.1 Controls & presets

- Density 0–100% fill, with randomized no-show (the existing no-show mask concept generalizes here).
- Color variation per person, plus supporter sections: home/away color blocks mapped to stands via the existing stand-bucketing (E/N/W/S).
- Atmosphere presets: “Sold-out derby,” “Half-empty midweek,” “Away end only,” “Neutral final.”
- Crowd-vs-tifo toggle: show the crowd only on non-tifo seats (so the choreography reads cleanly), or fill the whole stadium. This pairs with the banners/flags non-painted-area control in Section 11.

Impostors are effectively free at scale (one instanced draw for tens of thousands), so a packed bowl is achievable at HIGH; 3D figures and animation are layered in for near stands at ULTRA with level-of-detail and culling, and density auto-scales under the performance manager.

11. Banners & Flags

11.1 Banners

- Types: rail banners (on tier railings), hanging banners (draped from a balcony), tier banners, corner banners, and massive whole-stand banners.
- Custom uploads: a user image becomes a banner texture, reusing the existing image-import pipeline.
- Placement: anchor lines derived from the stand geometry (a rail line per tier, corners, stand centers) so banners snap to believable positions.
- Large fabric banners can use light cloth simulation on the edges (ULTRA) for a draped look; smaller ones are simple textured planes (HIGH).

11.2 Flags

- Small and large flags on poles as textured quads; animated / ultra-style waving via a cloth or vertex-shader wave.
- Crowd flags (thousands of small waving flags) are instanced and shader-driven — an ULTRA effect.

Headline UX control: non-painted areas

The user decides whether banners and flags automatically populate areas that are NOT part of the tifo. This lets empty or non-design stands gain match-day atmosphere without ever covering the choreography — a toggle plus a density slider, scoped to non-painted seats/stands.

12. Choreography Elements

- Card displays: the existing core; extend with per-seat card vs mosaic modes.
- Fabric tifos / giant overhead banners: a large surface (optionally cloth-simulated) that unfurls or rolls down over a stand — the classic overhead tifo.
- Choreography waves: timed reveals across seats. The existing `applyReveal` contract and the `export reveal-clock` already do this; extend them to sequence across stands.
- Multi-stage and layered displays: a timeline of reveal operations — cards flip, then the overhead banner drops, then flags raise — across elements and stands.
- Reveal animations: reuse the existing reveal contract so a sequence plays identically in editor preview, simulator, and exported video.

This is delivered as a lightweight **timeline** layered on the existing reveal mechanism; a visual timeline editor can follow once the playback engine exists.

13. Match Day Effects

Effect	Technique	Perf impact	Complexity	Tier
Smoke / colored smoke	GPU particles (alpha/additive), per-emitter tint	Med	Med	HIGH/ULTRA
Pyrotechnics (flares, sparks)	Particles + bloom (visual only)	Med–High	Med	ULTRA
Confetti	Instanced quads, physics-lite	Med	Med	ULTRA
Streamers	Ribbon geometry	Med	Med	ULTRA
Floodlights	Corner masts, emissive lamps, light cones	Med	Med	HIGH
Stadium shadows	Shadow map from sun/floodlights	Med–High	Med	HIGH
Post-processing	Bloom + SSAO + tone mapping + AA (EffectComposer)	High	Med	ULTRA
Day/night + dusk sky	Sky shader / gradient + light color	Low	Low	HIGH

13.1 Feasibility, performance, complexity

- Feasibility: all are standard, well-trodden Three.js techniques (GPU particles, shadow maps, EffectComposer). Nothing here is research-grade.
- Performance: floodlights + one shadow light + a dusk sky deliver the day-to-night transformation at HIGH for modest cost. Post-processing and dense particles are the expensive layer — hence ULTRA, with per-effect budgets and auto-scaling.
- Complexity: individually medium; the integration work (a shared particle/effect manager and an effects timeline) is the larger task and is sequenced late, after structure, crowd, and cameras.

14. Viewing Modes & the Camera Director

Today's five static presets become a Camera Director: a set of named shots with position, target, field-of-view, and optional motion. Several shots can be placed at real seat positions straight from SeatMap.pos3.

Mode	Implementation	Notes
TV Broadcast	High gantry, broadcast framing, slight follow	Builds on today's TV-gantry preset
Main Camera	Halfway-line elevated	Classic broadcast main
Pitch View	Pitch-level low angle	Player's-eye drama
Goal Camera	Behind-goal elevated	Great for end-stand tifos
Tunnel View	From the tunnel mouth looking out	Walk-out atmosphere
Crowd View	Placed at a real seat (pos3)	A supporter's actual sightline
Ultra View	From the painted stand, looking across	The designer's intended view
Drone View	Free elevated tracking	Exploratory
Cinematic Flyover	Scripted spline path (orbit, descend, push-in)	Hype-reel; renders to MP4/GIF via existing export

The flyover reuses the existing frame-capture export pipeline (renderOnce + readable buffer), so a cinematic match-day hype reel of the tifo becomes a one-click output — a strong shareable asset.

15. Competitive Analysis

- Sports visualization / broadcast AR tools: realistic but built for live data overlays, not for designing or simulating fan choreography.
- Stadium planning / CAD-BIM software: architecturally accurate but heavyweight, expensive, and not about the tifo or the crowd experience.
- Fan choreography planners: mostly 2D mockups or static image overlays — none offer a realistic, seat-accurate 3D match-day simulation tied to a live design.

The moat

The advantage is the integrated pipeline: seat-accurate design (already shipped) -> realistic match-day simulation (this proposal) -> cinematic, shareable output (an upgrade of the existing export). No competitor combines all three on real bowl geometry with a filled crowd and atmosphere. That integration — not any single effect — is what makes this the best-in-class choreography visualizer.

Note on likeness: as with the community-stadium policy, keep venues, crests, and sponsors generic unless the user supplies their own assets. Realism should never imply official affiliation with real clubs or stadiums.

16. Development Phases & Estimated Complexity

Each phase ships a visible win, lives in the simulator module, and leaves the editor untouched. Complexity is relative engineering effort (S, M, L, XL), not a calendar estimate.

Phase	Scope	Complexity	Primary risk
0. Foundation	Lazy-loaded simulator scaffold, shared-data interfaces, quality manager, capability probe, dispose discipline	M	Boundary design
1. Stands + lighting	Extrude bowl shell + roof from template, PBR concrete, sun + 1 shadow light, dusk sky	L–XL	Geometry extrusion correctness
2. Crowd	Instanced impostor crowd, density, sections, presets, crowd-vs-tifo toggle	L	Mobile fill performance
3. Pitch-side realism	Pitch texture/stripes, dugouts, tunnel, LED boards, technical area	M	Asset volume
4. Banners + flags	Placement system, custom uploads, flags (static then waving), non-painted-area control	M–L	Cloth perf at ULTRA
5. Effects + atmosphere	Floodlights, smoke/colored smoke, confetti/streamers, pyro, post-processing	L–XL	Perf budget / auto-scaling
6. Cinematic cameras	Camera Director, all viewing modes, flyover spline, MP4/GIF hype-reel export	M	Export integration
7. Choreography timeline	Multi-stage / layered reveals, overhead-banner unfurl, wave sequencing	M–L	Timeline UX

Phases 1 and 5 are the heavy lifts (stand extrusion; effects + post-processing). Phases 0, 3, and 6 are comparatively contained. Sequencing structure and crowd first delivers the biggest perceived realism per unit of effort.

17. Performance & Visual Quality Expectations

17.1 Performance targets

- Editor (LOW/MEDIUM): unchanged — 60 fps at 80k seats, mobile-friendly. This is a hard guarantee of the dual-renderer split.
- Simulator HIGH: 50–60 fps on desktop, ~30 fps on mid-range mobile — PBR, shadows, impostor crowd, floodlights.
- Simulator ULTRA: 30–60 fps on high-end desktop — post-processing, particles, waving flags — auto-downscaling on weaker GPUs.
- Techniques to hit targets: instancing everywhere, LOD on crowd and structure, frustum culling of real meshes, a single shadow cascade, capped device pixel ratio, particle budgets, lazy asset loading, and full disposal on exit.

17.2 Visual quality

The jump is from “colored cards on a flat pitch” to “a believable floodlit bowl at dusk, packed with a color-coordinated crowd, the tifo blazing across the ultras' end, flags waving and smoke drifting, shot from a broadcast gantry.” The look is clean, modern, web-real — PBR, shadows, and bloom carry most of the perceived-quality gain; the crowd and atmosphere carry the emotional impact.

18. Risks & Mitigations

Risk	Mitigation
Low-end / mobile performance	Auto-scaling + guaranteed LOW fallback (the editor renderer always works).
Bundle size growth	Simulator is a separate lazy-loaded chunk; compressed (KTX2) assets loaded only on entry.
Scope creep	Strict phasing; each phase is shippable; ULTRA effects are optional.
Maintaining two renderers	One engine (Three.js), shared data layer; the editor renderer stays tiny and frozen.
Asset pipeline overhead	Small curated set; generate crowd impostors and simple geometry procedurally where possible.
WebGL context leaks	Enforce the existing dispose discipline on simulator exit; only one heavy context at a time.
Club / venue likeness	Keep generic; carry the non-affiliation disclaimer; real crests/sponsors only if user-supplied.
Mobile GPU limits (shadows, particles)	Tier-gate heavy features; validate on real devices; conservative Auto defaults.

19. Long-Term Roadmap

Beyond the seven phases, the platform can keep extending the “design -> simulate -> film -> share” loop:

- WebGPU backend swap for additional headroom (behind the renderer abstraction from Section 6).
- Volumetric lighting and fog for floodlight beams and atmospheric depth.
- Real cloth simulation for hero tifos and giant banners.
- Reactive crowd: audio ambience plus animation that responds to reveals and goals.
- “Stand mode” in WebXR — stand in your own end and watch the choreography in VR.
- AI-directed camera: auto-generate a hype reel from the design and stand layout.
- Shareable simulator scenes and, longer term, multi-user stadium walkthroughs.

20. Appendix: Data Contracts & Module Layout

20.1 Shared data (reused unchanged or extended)

- SeatMap: count, uv, pos3, tierOf, rowOf, sectionOf, neighbors, mirrorOf, bounds, templateRef — consumed by both renderers.
- DesignStore: cells, palette, onDirty(indices), onPaletteChange — the live design + dirty events.
- StadiumTemplate (extended, backward-compatible): + trackWidthMeters, + per-stand overrides, + roof config, + pitch-side furniture flags.

20.2 Proposed module layout

- src/render/preview3d.ts — EDITOR renderer (unchanged; LOW/MEDIUM).
- src/render/simulator/index.ts — lazy entry point + lifecycle (mount/dispose).
- src/render/simulator/scene.ts — scene graph, lights, sky, post-processing pipeline.
- src/render/simulator/stands.ts — extruded bowl shell, roof, stairs, pitch-side furniture.
- src/render/simulator/crowd.ts — instanced impostors / LOD figures, density, sections.
- src/render/simulator/banners.ts — banners + flags placement, custom uploads, non-painted-area logic.
- src/render/simulator/effects.ts — particle + effect manager (smoke, pyro, confetti).
- src/render/simulator/cameras.ts — Camera Director, viewing modes, flyover spline.
- src/render/simulator/quality.ts — capability probe + auto-scaling manager.
- src/render/simulator/assets.ts — lazy, compressed texture/asset loading.

Bottom line: protect the editor by freezing today's renderer as the fast path, and invest all new realism into a separate, lazy-loaded Match Day Simulator on the same engine and data. Sequence stands and lighting first for the biggest immediate leap, then crowd, pitch-side detail, banners and flags, effects, and cinematic cameras — arriving at the most realistic, immersive fan-choreography visualizer available, with editing as fast as it is today.